Linux 离线安装包（flocks-offline.run）

给装机时不能联网的 RHEL 8 / 9 系机器（glibc ≥ 2.28）用的单文件安装包，在 RHEL 8.10（UBI 8.10）容器里构建。客户只需要 `scp` 上去、`sudo bash /var/tmp/flocks-offline.run`，装完浏览器开 `http://服务器IP:5173`。包里带齐 Python 3.12、Node.js、uv、预建好的 `.venv`、构建好的 WebUI，安装过程不需要 PyPI、npm、GitHub。想整体放到数据盘上，就用 `sudo FLOCKS_OFFLINE_BASE=/data bash /var/tmp/flocks-offline.run`，程序、数据、配置、日志全进 `/data/flocks`，见装到指定目录。变量要写在 `sudo` 后面：写成 `FLOCKS_OFFLINE_BASE=/data sudo bash …` 会被 sudo 的 `env_reset` 清掉，照样装到 `/opt/flocks`。

包里带不带 Pro bundle，决定装好之后怎么升级。默认不带：装出来是开源版，升级和普通源码安装一样走网络，页面上登录云账号、审批通过后点「开始升级」装 Pro，核心更新走「检查更新」；`flocks.env` 里写的是 `FLOCKS_DEPLOY_MODE=source`，没有任何离线开关。构建时加 `--flockspro-wheel` 就是带 Pro bundle 的包，整机离线：Pro 从包内 `/opt/flocks/bundle` 装，uv / npm 固定离线，联网升级一律拒绝，核心升级靠新版 `.run` 覆盖。

这套东西和 `install.sh` / Docker / Windows 包互不影响：装到机器上的形态还是"仓库根 + `.venv` + `webui/dist`"，只是放在 `/opt/flocks/flocks`、以 `flocks` 用户跑、由 `/etc/flocks/flocks.env` 提供环境变量。代码里为离线升级加的逻辑全部由 `FLOCKS_DEPLOY_MODE=offline`、`FLOCKS_PRO_BUNDLE_DIR`、`FLOCKS_UPDATE_CHANNEL` 和 bundle manifest 的 `prebuilt: true` 触发，只有带 Pro bundle 的包会设置这些。

内容

- 文件
- 怎么构建
- 装到机器上是什么样
- 旧数据怎么带过来
- 装到指定目录（FLOCKS_OFFLINE_BASE）
- Pro 升级怎么走
- 和原来的升级方式是什么关系
- 测试
- 在 Docker 里模拟客户内网

文件

文件	作用
`build-in-docker.sh`	一键打包：起 `versions.manifest.json` 里 `build_image` 指定的 RHEL 8.10（UBI 8.10）容器跑下面的构建脚本，`--arch x86_64
`export-source.sh`	从 git 导出干净源码树（`git archive HEAD` + 锚定在仓库根的 exclude pathspec，去掉 tests / packaging / docs 等顶层开发目录），核对导出文件数等于 git 列出的数，再对结果做敏感文件复检；`build-offline-run.sh` 与 `build-in-docker.sh` 都靠它拿源码
`build-offline-run.sh`	真正的构建脚本，在 `build_image` 容器里跑，产出 `flocks-offline.run` + `.sha256` + `versions.json`
`glibc_floor.py`	读包里每个 ELF 文件的符号版本需求（`.gnu.version_r`），要的 glibc / libstdc++ 比 `glibc_floor`（RHEL 8：`GLIBC_2.28`、`GLIBCXX_3.4.25`、`CXXABI_1.3.11`）新就让构建失败，ELF 文件读不全（表被截断、链接或偏移越界、没有节头的动态链接文件）也算失败；`smoke-test.sh` 装完后在容器里再查一遍
`versions.manifest.json`	锁定的 Python / Node / uv / makeself 版本与下载地址模板、构建镜像 `build_image` 和兼容下限 `glibc_floor`；升级工具链改这里
`installer/install.sh`	打进包里的安装脚本，`sudo bash xxx.run` 时由 makeself 自动执行，全程不提问
`installer/flocks.env.template`	渲染成 `/etc/flocks/flocks.env`
`installer/flocks.service`	渲染成 `/etc/systemd/system/flocks.service`
`installer/flocks-cli-wrapper.sh`	渲染成 `/usr/local/bin/flocks`，`sudo flocks status` 会以服务用户身份执行；重跑安装包时也靠它认出上次装在哪

文件	作用
`installer/relocate.py`	构建时 `scan` 出 venv 和 `tools/` 里写死构建路径的文件，写成 `installer/relocate.list`；装到别的目录时 `apply` 按清单改写
`smoke-test.sh`	在模拟的客户内网里（见下文「在 Docker 里模拟客户内网」，`SMOKE_NETWORK=none` 改成完全无网络）用一次性容器装一遍（镜像默认是 `build_image`，也就是支持的最老系统），按包里有没有 Pro bundle 走两套检查。共同部分：包里所有二进制过 `glibc_floor`、cryptography 的原生模块能加载、env 里有 `UV_NO_SYNC=1`、以服务用户和服务环境执行技能用的 `uv run python -c ...` 不触发重建和下载、装在非默认端口（默认 5273，`SMOKE_PORT` 可改）、建管理员、不带变量重跑安装包要沿用端口、`SMOKE_INIT=systemd` 时验 systemd / firewalld / 模拟重启、`FLOCKS_OFFLINE_SKIP_START=1` 只装不启动；`SMOKE_BASE=/data` 时整套检查换成基目录布局跑（基目录挂一个单独的卷，像独立数据盘），另外验 `/opt/flocks` 下什么都没建、venv 里不再有 `/opt/flocks`、不带变量重跑能找到原目录、目录不存在 / 在 `/var/tmp` 下 / 换到另一个基目录都被拒绝。开源版的包：env 是 source 模式且没有离线开关；本地 console 替身下 Pro 升级越过离线拦截去下载 bundle（离线 env 作对照、必须被拒）；重跑把离线 env 切回在线；在线升级过 Pro 的实例拒绝被开源包覆盖；已装版本更新时拒绝覆盖、上次在线升级没完成时放行恢复并改回版本标记。带 Pro bundle 的包：本地 Pro 包安装、离线模式拒绝联网升级、不带 Pro 的新包覆盖已激活实例（沿用 / 拒绝两种，拒绝那次核对没有新的 `.bak` 目录且解包目录被挪到 `.failed-*`）、`SMOKE_INIT=systemd` 时重启容器验 Pro 存活与真 firewalld 永久规则、降级到 OSS 的 prebuilt 重启（放在重启之后，因为重启那段要求 Pro 还在）、`FLOCKS_OFFLINE_SKIP_START=1` 只装不启动，输出 PASS/FAIL

文件	作用
`intranet-sim.sh` / `intranet-allowlist.txt`	模拟客户内网：被测容器只能访问离线安装手册里给服务器放行的地址，其余域名解析不了、其余地址和端口连不上
`verify-install.sh`	在装好的真机上跑的验收脚本（root）：布局（从 `/usr/local/bin/flocks` 读出装在哪，默认布局和基目录布局都认）、systemd、firewalld、建管理员/登录、本地 Pro 包安装、`--rerun` 重复执行安装包，输出 PASS/FAIL。`--port` 默认读机器上的 `FLOCKS_PORT`（`--rerun` 会把它作为显式端口传给安装脚本，写死 5173 会把 5273 的实例悄悄挪走）；不传 `--admin-pass` 时用包内 Python 的 `secrets` 生成随机密码（原来的 `tr </dev/urandom | head -c 16` 在 `pipefail` 下必定 exit 141）

CI：`.github/workflows/linux-offline-run.yml` 在 x86_64（`ubuntu-latest`）和 aarch64（`ubuntu-24.04-arm`）两个原生 runner 上各起一个 RHEL 8.10（UBI 8.10）容器构建，手动触发上传两个 artifact（可填 Pro wheel 的 URL 和 release id），发布 Release 时把 `flocks-offline-x86_64.run`、`flocks-offline-aarch64.run` 挂到 Release 资产；交付给客户时改名成说明书写死的 `flocks-offline.run`。

怎么构建

一键（本机有 docker 即可）：

```
packaging/linux/build-in-docker.sh --arch all          # 开源版，装好后在线升级（默认）
# 产物：packaging/linux/Output/x86_64/flocks-offline.run、
        packaging/linux/Output/aarch64/flocks-offline.run
packaging/linux/build-in-docker.sh --arch all --flockspro-wheel
        ../flockspro/dist/flockspro-<ver>-py3-none-any.whl   # 带 Pro bundle，整机离线
```

`--arch` 默认是本机架构；另一种架构要 Docker 里有 QEMU（`docker run --privileged --rm tonistiigi/binfmt --install all`），慢好几倍，同架构机器或 CI 矩阵更快。Apple Silicon 上用 Colima 的话，`colima start --vm-type vz --vz-rosetta` 走 Rosetta 跑 x86_64 容器，比 QEMU 快得多。国内机器加 `--cn` 走 npmmirror。`--keep-container` 会留下容器，进去 `build-offline-run.sh --repack` 可以只换脚本快速重打。

为什么在 RHEL 8.10 里构建：`uv sync` 按构建机的 glibc 挑 wheel。之前在 CentOS Stream 9（glibc 2.34）上构建，cryptography 46.0.5 选中了 `manylinux_2_34` 的 wheel，装到 RHEL 8 系（glibc 2.28）的机器上，`import cryptography` 直接报 `GLIBC_2.30 not found`（Rocky 8.9 实测），依赖它的企业微信、微信渠道和 SSH / JWT 相关功能都会受影响；uv 还会想换成 2_28 的 wheel，离线下载失败。在 glibc 2.28 上构建，uv 只会选 2.28 以内的 wheel。构建最后还会用 `glibc_floor.py` 把 `tools/` 和 `flocks/`（含 `.venv`）里每个 ELF 文件的版本需求和 `glibc_floor` 比一遍，超了就失败，所以换 `--image` 到更新的系统上构建会直接报出是哪个文件。同一个 x86_64 包在 RHEL 8.10（UBI 8.10，带 systemd + firewalld）和 CentOS Stream 9 的断网容器里都跑过 `smoke-test.sh`。

源码取自 **`git archive HEAD`**，不是工作区：有未提交改动时脚本直接拒绝（`versions.json` 里的 `git_sha` 要能对上真实提交），本地调试可以加 `--allow-dirty`，这时会把工作区里已跟踪 + 未忽略的文件一起导出，`versions.json` 记 `source_dirty: true`。`build-in-docker.sh` 在本机导出到 `~/.cache/flocks-offline-build/<arch>/source/` 再只读挂进容器（上一次的导出改名为 `source.previous-<时间戳>` 留着，攒多了自己清），容器里没有工作区可 tar。排除用的是 git pathspec（`:(exclude)tests` 这种，只认仓库根那一层），不是 `tar --exclude`：bsdtar 的 exclude 不锚定，`./assets` 会把 hub 技能里嵌套的 `assets/` 一并吃掉——第一次在 macOS 上导出就丢了 279 个文件，包装上能跑，谁也没发现，所以现在导出后还会核对文件数（导出数 ≠ git 列出的数直接失败）。无论哪种模式，导出后都会复检一遍：`.env`、`.secret.json`、`*.pem` / `*.key`、`id_rsa*`、`*.log`、`.flocks/{config,data,run,logs,workspace}` 和本地的 `flocks.json` / `mcp_list.json` 只要出现就构建失败（`*.example` 除外）。之前是 `tar` 整个工作区，会把 `.flocks/.storage_migrated` 这类本地状态、甚至 `.env` 一起打进客户的包里。

venv 里写死了建它的路径（`pyvenv.cfg`、shebang、editable `.pth` 全是绝对路径），所以构建在默认安装路径 `/opt/flocks` 下进行，要求这个目录为空——这就是一定要在干净容器里跑的原因。装到别的目录靠安装时按 `relocate.list` 改写，见下面“装到指定目录”。手动等价命令：

```
docker run --rm -it -v "$PWD":/src -v ~/.cache/flocks-offline-
        build:/root/.cache/flocks-offline-build \
  registry.access.redhat.com/ubi8/ubi:8.10 bash -c '
    dnf -y install tar gzip findutils which git python3 diffutils procps-ng
        iproute &&
    cd /src && bash packaging/linux/build-offline-run.sh --output-dir
        /root/.cache/flocks-offline-build/out'
```

带 Pro bundle：加 `--flockspro-wheel /path/flockspro-<ver>-py3-none-any.whl`，可选 `--pro-release-id`（console 上对应的 release）、`--pro-bundle-version`。wheel 必须是和这个 core 配对的版本。不加就是开源版的包，Pro 组件不进包，客户审批通过后在页面上在线升级。

国内构建机可以走镜像：`FLOCKS_NODE_BASE_URL=https://registry.npmmirror.com/-/binary/node`、`FLOCKS_NPM_REGISTRY=https://registry.npmmirror.com/`、`FLOCKS_UV_DEFAULT_INDEX=https://mirrors.aliyun.com/pypi/simple`。下载的工具链缓存在 `~/.cache/flocks-offline-build`，改 manifest 才会重新下。

构建做的事：下载工具链 → `export-source.sh` 导出源码（`git archive`，不含 `.git`、tests、packaging、docs、`node_modules`）并复检 → 建完包 `python3 ~/.claude/tools/artifact_scan.py Output/<arch>/flocks-offline.run --repo .` 再看一眼包里有没有不该有的东西 → 用包内 Python 跑 `uv sync --frozen --group dev` → 在 scratch 目录 `npm ci && npm run build` 后只把 `dist` 拷进来 → `glibc_floor.py` 查一遍要发出去的二进制 → 写 Pro bundle 和 `versions.json` → `relocate.py scan` 生成 `installer/relocate.list`（venv 和 `tools/` 里有二进制文件带 `/opt/flocks` 就构建失败）→ makeself 打包（`--target /opt/flocks/.staging`，解包不占 `/tmp`；用的是补过一段的头部，设了 `FLOCKS_OFFLINE_BASE` 就改解到基目录下，打完核对这段确实在 `.run` 里）。x86_64 和 aarch64 都能构建，产物只能装到同架构机器。

装到机器上是什么样

`/opt/flocks/tools/{python,node,uv}`	运行时，root 所有
`/opt/flocks/flocks`	仓库副本 + .venv + webui/dist，flocks 用户所有（升级要写它）
`/opt/flocks/bundle`	Pro bundle（manifest.json + wheels/），有才装
`/opt/flocks/cache/uv`	构建机的 uv 缓存，UV_OFFLINE=1 时兜底
`/opt/flocks/installer, versions.json`	这次安装用的脚本和版本信息
`/etc/flocks/flocks.env`	服务环境变量；改 IP/端口/控制台地址就改它然后 systemctl restart flocks
`/etc/systemd/system/flocks.service`	oneshot + RemainAfterExit, ExecStart=flocks start, ExecStop=flocks stop
`/var/lib/flocks/.flocks`	数据（配置、数据库、日志、授权）；flocks.env 里改过 FLOCKS_ROOT 就是改过的那个。已有数据时安装脚本不动它，第一次装会把旧实例的数据拷进来（见下一节）
`/usr/local/bin/flocks`	CLI 包装

安装脚本会：先做预检（架构、磁盘、端口、已激活 Pro 与 bundle 的匹配），预检不过就报错退出、不动任何文件，并把已解包的安装文件（只挪 `versions.json`、`installer/`、`tools/`、`flocks/`、`bundle/`、`cache/` 这几项，解包目录本身不改名——`--target /var/tmp` 时那是共享的系统目录）收进 `<解包目录>/flocks-offline-failed-<时间戳>/`，makeself 失败时不清

`--target` 目录，不收起来的话下一个包会解在它上面、两个版本的文件混在一起；下次装成功时这些残留会被搬到 `/opt/flocks/` 下和 `*.bak-*` 放一起。任何没被显式处理的命令失败也走同一条路（ERR trap），日志里会写是哪条命令、第几行——`$(...)` 里出错只退那个子 shell，由父进程报外层命令，`fail()` 的输出全走 stderr，不会被命令替换当成变量值；架构核对在用到包内 Python 之前就做（异架构包的 Python 根本起不来，否则只能看到 Exec format error）；建 `flocks` 用户；已有安装先停服、把旧的 `flocks/`、`tools/`、`bundle/` 改名成 `*.bak-<时间戳>`（不删）；搬入新文件后 `restorecon -R`（从临时目录 mv 过来的文件会带 `tmp_t` 标签，SELinux enforcing 下 systemd 起不来）；合并 `flocks.env`（受管的几个键覆盖，其它保留）；如果之前激活过 Pro（`~/.flocks/run/pro-bundle-installed.json` 在），把包里的 Pro wheel 重新装进新 venv；新包不带 `bundle/` 时，先看机器上原来的 `/opt/flocks/bundle` 能不能沿用（`manifest.json` 的 `core_version` 等于新包 core、wheel sha256 对得上 manifest），能就原样搬回来继续装 Pro，不能就在动任何文件之前报错退出（“这台机器已激活 Flocks Pro……本次未做任何改动”；机器上没有 bundle 时看 env：离线实例提示用带 Pro 组件的配套安装包，Pro 是在线升级装的则提示在页面上在线升级），不会把一台 Pro 实例静默装成 OSS；`systemctl enable --now`；firewalld 放行服务端口（默认 zone 加上所有绑了网卡/来源的 zone，永久规则和运行态规则分别核对补齐——之前只查运行态，运维手动 `--add-port` 过一次的机器永久规则就永远不会写，reload 或重启后端口关闭；firewalld 拒绝写入只打警告、给出手动命令，不会让已经换好文件的安装中途退出）；`chronyd` 打开；轮询 `/api/health` 直到 200；最后打印 `[flocks] 安装完成：http://<IP>:<端口>`。日志在 `/var/log/flocks-offline-install.log`。

`flocks.env` 按机器上最终有没有 Pro bundle（包里带的，或沿用的）来写。有 bundle 写 `FLOCKS_DEPLOY_MODE=offline` 和一组离线开关：`FLOCKS_OFFLINE_INSTALL`、`FLOCKS_UPDATE_CHANNEL`、`FLOCKS_PRO_BUNDLE_DIR`、`UV_OFFLINE`、`npm_config_offline`。没有 bundle 写 `FLOCKS_DEPLOY_MODE=source`，这组开关一行都不留，连之前装带 Pro 的包时留在文件里的也删掉：`UV_OFFLINE` / `npm_config_offline` 会让在线升级的依赖同步失败，离线渠道会让 console 找不到 release，`FLOCKS_OFFLINE_INSTALL=1` 则在 `FLOCKS_DEPLOY_MODE` 被删掉时把 updater 重新判成离线实例。`FLOCKS_DEPLOY_MODE` 是受管键，所以同一台机器换另一种包重跑，升级方式跟着切过去。dry-run 会打出 `upgrade=online|offline`。

能在线升级以后，机器上的版本可能比手里的 `.run` 新（在页面上升过核心，或者 Pro bundle 带了更新的核心）。安装脚本在预检时读 `/opt/flocks/flocks/pyproject.toml` 的版本，比安装包新就在动文件之前拒绝，提示换用不低于它的安装包，免得旧代码跑在新版本用过的数据上；同版本重跑（修复）照常。有两种情况放行：一是上次在线升级到的正是这个版本、而且没做完（`/var/lib/flocks/.flocks/run/upgrade-result.json` 的 phase 是 failed 或 running），安装包又不比升级前跑的版本（记录里的 `current_version`）旧——handoff 先换源码再跑 `uv sync` / npm，失败不回滚，这时重跑手里的 `.run` 就是恢复办法，phase 是 running 或卡在 start 时会多打一条"新版本可能已经启动过"的警告；二是运维显式加

`FLOCKS_OFFLINE_ALLOW_DOWNGRADE=1`。每次装完，这份升级记录都会改名成 `.bak-<时间戳>`，不会被第二次拿来放行；updater 的版本标记 `.flocks/version/.current_version` 只要比装好的代码新，就改回安装包的版本，不然页面会报一个没装的版本号。

端口只有一个来源：本次显式给的 `FLOCKS_OFFLINE_PORT` > 机器上 `/etc/flocks/flocks.env` 里已有的 `FLOCKS_PORT` > 5173，env 文件、firewalld、健康检查和最后那行 URL 用的都是它。所以装在 5273 上的实例，之后直接 `sudo bash 新包.run` 不带变量也还是 5273（之前会退回 5173 去查健康，等 180 秒后报“没有就绪”）；要换端口就显式传 `FLOCKS_OFFLINE_PORT`，这时 env 里的 `FLOCKS_PORT` 才会被改写（env 里那行本身是坏值时也会被改写成 5173，否则服务起不来）。装好后留在 `<安装目录>/installer/` 的脚本副本只供查看，直接执行会被拒绝——它的解包目录就是安装目录，跑起来会把自己所在的安装挪成 `.bak`。认副本主要看位置：解包目录是构建路径、是 wrapper 或 unit 指的安装目录、或者就是这次要装进去的目录；另外 venv 的 `pyvenv.cfg` 已经指向别处（装到基目录的实例改写过它）也算，这样 wrapper 丢了、venv 坏了都认得出来。认出来之前出错一律不收拾解包目录，免得把正在跑的程序挪走。

`flocks.env` 里的 `UV_NO_SYNC=1` 是给智能体用的：技能让它用 bash 跑 `uv run python -c ...`，这个子进程继承服务的环境。不加这一项，`uv run` 会先同步项目：包解压到客户机器上时文件的 ctime 变了，uv 认定可编辑安装的 flocks 过期、要重建，重建要从 PyPI 拉 hatchling，离线必失败，Rex 的配置预检就报 `Failed to download`。加了之后 `uv run` 直接用包里的 `.venv`。它同样作用于 `uv add` / `uv remove`：只改 `pyproject.toml` 和 `uv.lock`，不往 `.venv` 里装或卸包（离线机器上本来也拉不到新包，要加依赖就在联网机器上重打安装包）；显式的 `uv sync`、`uv pip install`（升级、Pro 安装走的路径）不受影响。这一项不在受管列表里，老机器重跑新包时 env 合并会把它补上，运维改过的值不覆盖。

机器上没有 lsof / fuser 时（最小化安装常见），`flocks start` 只能靠试着 bind 来判断端口有没有被占。之前这个试探不带 `SO_REUSEADDR`，停服后浏览器连接（开着的 WebUI 页面一直挂着事件流）在服务端留下的 TIME_WAIT 会让它在一分钟内都报“端口已被占用，但当前环境无法识别占用 PID”：开着 WebUI 重跑 `.run`、`systemctl restart flocks`、Pro 激活后的重启都会失败。现在 Linux 上的试探和 uvicorn 一样带 `SO_REUSEADDR`，TIME_WAIT 不再算占用，真有进程在监听时照样报。

systemd 用 oneshot 而不是把 `service-daemon` 放前台，是因为在线升级的 `restart_handoff` 会先 `stop_all` 再自己拉起新的 `flocks start`；前台模式下主进程一退 systemd 会连 handoff 一起杀掉。oneshot 下 `systemctl status` 只能看到 active (exited)，看进程用 `sudo flocks status`。

调试用的环境变量（客户不需要）：`FLOCKS_OFFLINE_DRY_RUN=1` 只打印计划；`FLOCKS_OFFLINE_SKIP_SYSTEMD=1` 在容器里用 `flocks start` 直接起；`FLOCKS_OFFLINE_SKIP_FIREWALL=1`、`FLOCKS_OFFLINE_SKIP_START=1`（只装不启动，smoke

里有真跑；之前这条分支调了一个还没定义的函数，exit 127）、`FLOCKS_OFFLINE_PORT`、`FLOCKS_OFFLINE_HEALTH_TIMEOUT`。

旧数据怎么带过来

数据目录先看机器上已有的 `/etc/flocks/flocks.env`：里面的 `FLOCKS_ROOT` 是绝对路径就一直用它（运维把数据挪到数据盘的情况），没有才用 `/var/lib/flocks/.flocks`。Pro 标记、建目录、`install_profile.json` 都跟着这个目录走。只认 `FLOCKS_ROOT=` 这种写法，和 systemd、安装脚本合并 env 的读法一致：`export FLOCKS_ROOT=` 对服务不生效，安装脚本会打警告并按默认目录处理；写成相对路径直接报错退出。之前这里写死了默认路径，数据挪过盘的机器重装时读不到 Pro 标记，会把 Pro 实例装成开源版。

数据目录里还没有用户数据时（`config/flocks.json`、`config/flocks.jsonc`、`data/flocks.db` 都不在，一般就是这台机器第一次装离线包），安装脚本会找上一个实例的数据拷一份过来，按这个顺序找：

1. `FLOCKS_OFFLINE_MIGRATE_FROM=<目录>`，显式指定；写 `none` 表示不迁移。
2. 启动安装脚本时带的 `FLOCKS_ROOT`，对应在线版改过数据目录的情况。
3. 本机用户默认的 `~/.flocks`：执行 sudo 的用户、root、`/etc/passwd` 里每个用户的 home。只有一个有数据就用它；有好几个就列出来（带数据库最后写入时间）报错退出，让运维用第 1 条指定，脚本不替人猜。

sudo 默认会清掉调用方的环境变量，变量要写在 sudo 后面：`sudo FLOCKS_OFFLINE_MIGRATE_FROM=/home/admin/.flocks bash /var/tmp/flocks-offline.run`。

数据目录本身也先过一遍：不能是 `/`；已经存在但没有用户数据时，里面只能有 Flocks 自己的目录（config、data、run、logs、plugins、workspace 这类），混着别的东西（别的服务的文件之类；新 ext4 盘根目录的 `lost+found` 不算）就当作数据目录配错了，报错退出，安装脚本不会去建目录、改属主或改名。显式给了 `FLOCKS_OFFLINE_MIGRATE_FROM`、数据目录却已经有数据时，也报错退出，不会悄悄跳过。

动文件之前还要查三件事：有进程还开着源目录里的文件（旧实例没停，按 `/proc/*/fd` 查），就报出 PID 和命令行后退出；数据目录是挂载点时没法整目录替换，让运维把 FLOCKS_ROOT 改到挂载点下的子目录；数据目录真实所在的分区（软链按链接目标算）放不下一份拷贝（另留 200 MB），也退出；旧目录顶层的外部软链指向的目录如果包含新数据目录（比如 `workspace -> /data` 而新数据目录在 `/data/flocks`），照拷会把拷贝本身再拷进去，同样拒绝。旧实例还占着端口时，端口检查会先报错，并提示先 `flocks stop`。迁移的数据里有 Pro 标记（`run/pro-bundle-installed.json`）就按已激活 Pro 处理：新包带 Pro 组件就装进新 venv，不带就在动

文件之前拒绝。旧目录里 updater 的版本标记（`version/.current_version`）比安装包新，说明那边在线升级过、数据已经被新版本用过，和覆盖安装的降级守卫一样在动文件之前拒绝，确认要迁就加 `FLOCKS_OFFLINE_ALLOW_DOWNGRADE=1`。

拷贝由 `installer/migrate_data.py` 用包内 Python 执行，源目录只读不写：

1. 整份拷到 `<数据目录>.migrating-<时间戳>`，跳过 socket、管道这类运行时文件。源目录顶层有指向别处的目录软链（比如 `data` 挪到了数据盘），拷成实体目录，新旧实例不共用同一份数据库。
2. 原来的数据目录如果已经存在（没有用户数据），把只有它有的文件并进拷贝，然后把它改名成 `.pre-migration-<时间戳>`。
3. 先把整份拷贝的属主交给 flocks，再把拷贝改名成数据目录。改名是最后一步，改名成功时数据已经是服务能用的状态。

这一步排在创建服务用户之后、停旧程序之前，拷失败时旧安装原样不动，已拷的部分留在 `.migrating-*` 里。中途断掉只有两种结果：没改名，目标原样，重跑会从头再迁一次；已改名，数据已经完整归 flocks 所有。改属主时软链只改链接本身，不会顺着它去改树外文件；已有实例的数据目录，安装脚本从不改它的属主。

旧实例里写死的绝对路径会跟着改：`config/flocks.json`、`config/flocks.jsonc`、`config/config.json`、`config/mcp_list.json`、`data/hub/**/*.json`（hub 安装记录的 `installPath`、`backupPath`）里以旧目录开头的字符串值，以及指向旧目录的软链，都换成新目录。改的时候只动字符串本身，注释、格式和键名原样保留；比较时先按 JSON 解码，`\uXXXX`、`\/` 这类转义写法也能匹配；读不了的文件（不是 UTF-8）会在安装日志里列出来。迁移记录写在 `run/offline-migration-<时间戳>.json`。旧目录不删，装完会提示确认后自己清。

有两处没有跟着改。工作流中心的注册表按工作流文件的绝对路径算 id，从 `/root/.flocks` 换到 `/var/lib/flocks/.flocks` 后，发布过的工作流 API 服务要重新发布一次，工作流定义本身已经拷过来了。在线版程序目录里的 `.env`（比如 `/root/flocks/.env`）不在数据目录下，不会带过来，里面的变量要搬进 `/etc/flocks/flocks.env`。

用 `FLOCKS_OFFLINE_BASE` 装的实例，上面说的 `/etc/flocks/flocks.env`、`/var/lib/flocks/.flocks` 分别是 `<基目录>/flocks/etc/flocks.env`、`<基目录>/flocks/data/.flocks`，找旧数据和拷贝的规则不变。

装到指定目录（FLOCKS_OFFLINE_BASE）

```
sudo FLOCKS_OFFLINE_BASE=/data bash /var/tmp/flocks-offline.run
```

设了基目录，所有东西都放到 `<基目录>/flocks` 下。基目录要事先建好或挂好。下面这几种情况，`.run` 在解压之前就退出，不会先解出一个多 G 的包再报错：基目录不存在或不是绝对路径；所在分区挂载了 noexec（程序没法从那里运行）；和默认安装目录嵌套（比如 `/opt`）；机器上已经有一个装在别处的实例，或者别处有一次没装完的安装（和安装脚本一样，先看 `/usr/local/bin/flocks`，再看 systemd unit，最后看默认目录，见下面“重跑”那段）。没装完的那种，提示里会列出旧实例留下的 wrapper、systemd unit 和 env 文件（只列带着安装脚本标记的）：确定不要旧实例了，把它们改名挪走再装。

`/data/flocks/app` 程序，对应默认布局的 `/opt/flocks`（tools、flocks、bundle、cache、installer、versions.json）
`/data/flocks/data` flocks 用户的 HOME，数据在 `.flocks` 下，对应 `/var/lib/flocks`
`/data/flocks/etc/flocks.env` 对应 `/etc/flocks/flocks.env`
`/data/flocks/logs/flocks-offline-install.log` 对应 `/var/log/flocks-offline-install.log`

留在系统目录里的只有 `/etc/systemd/system/flocks.service`、`/usr/local/bin/flocks`、`flocks` 用户和 SELinux 规则。解压也在基目录下（`/data/flocks/app/.staging`），`/opt` 下什么都不建：makeself 把 `--target` 写死在头部，构建时给头部补了一段，设了 `FLOCKS_OFFLINE_BASE` 就改指到基目录。

venv 是在 `/opt/flocks` 建的，写死这个路径的地方：`.venv/bin` 下 45 个入口脚本的 shebang 和 5 个 `activate*`、`pyvenv.cfg`、editable 安装的 `.pth` 和 `direct_url.json`、`bin/python` 软链，一共 53 个文件 1 个软链（x86_64 包的数；aarch64 包的入口脚本多两个）。构建时 `relocate.py scan` 把它们列进 `installer/relocate.list`；安装时在解压目录里按清单改写（只认完整路径，`/opt/flocks-old`、`/srv/opt/flocks` 这种不动），改完回读核对，这一步在停旧服务之前做，失败了运行中的实例不受影响；文件搬到位后再以服务用户跑 `python -I -c 'import flocks'` 和 `flocks --version`，通过了才往下走。`.pyc` 不用改，CPython 加载时会把里面的文件名换成实际路径。之后在线升级时 uv 自己会按新位置重写这些文件。

基目录的要求：绝对路径，只含字母、数字和 `._-/`，不超过 90 个字符（`#!<基目录>/flocks/app/flocks/.venv/bin/python` 要塞进老内核读的 127 字节）；不能在 `/tmp`、`/var/tmp`、`/run` 下（systemd-tmpfiles 会定期清），也不能在 `/dev`、`/proc`、`/sys`、`/boot` 下；不能和默认安装目录套在一起（`/opt` 就不行，会变成 `/opt/flocks/app`）。服务用户要能进基目录和它上面每一层，进不去就在动文件之前报错，提示

`chmod o+x`。`<基目录>/flocks` 会被改成 755（makeself 建它时用的是 umask 077）。`flocks` 用户已经存在且 HOME 不是 `<基目录>/flocks/data` 时不改它，服务按 `flocks.env` 里的 `HOME` 跑，日志里提一句。

重跑（升级、修复）照样带上同一个变量，客户手册也是这么写的。忘了带也能装对：安装脚本从 `/usr/local/bin/flocks`（没有就看 systemd unit）读出上次装在哪，按原目录升级，只是这时安装包先解压到默认的 `/opt/flocks/.staging`，要占 `/opt` 所在分区 1.4G 左右——选 `/data` 的机器往往就是系统盘小，空间不够时 makeself 在解压前就报错退出。装完会把这个空目录连同空的 `/opt/flocks` 一起删掉，日志里提示下次带上变量。这时解压目录的 noexec 检查管不到基目录，所以安装脚本会再查一次基目录所在分区，后来被重新挂成 noexec 的，在停旧服务、挪旧程序之前就拒绝，免得升级停在一半。已有的实例不会被搬走：带着另一个基目录重跑，或者机器上是默认布局这次却带了基目录，都在动文件之前拒绝（“安装包不会把它搬到……升级请去掉 FLOCKS_OFFLINE_BASE 重跑”）。升级做到一半断掉（旧程序已经改名成 `*.bak-*`、新程序还没搬进去）时，wrapper 指的目录里没有程序，但 env 文件和 `.bak` 目录还在，这种算“没做完”，重跑会在原目录接着装；wrapper 指向的目录里什么都没有了（删掉或改名了）才当没装过。

SELinux：`/data` 这类目录在策略里没有文件上下文，下面的文件都是 `default_t`，systemd 未必能执行或读取。开着 SELinux 时安装脚本用 `semanage fcontext -a -e` 把 app、data、etc、logs 分别登记成和 `/opt`、`/var/lib`、`/etc`、`/var/log` 等同，再 `restorecon -R <基目录>/flocks`，标签和默认布局一样，整机重打标签也不会丢。机器上没有 `semanage`（`policycoreutils-python-utils` 没装）时用 `chcon --reference` 直接打，当下能用，整机重打标签后会丢，日志里有警告；装上那个包再跑一次安装包就会登记成永久规则。SELinux 关着时只做原来的 `restorecon`；之后才打开 SELinux 的话，重跑一次安装包把规则补上。

Pro 升级怎么走

开源版的包装出来的实例，Pro 升级和普通源码安装完全一样：页面上"开始升级"调 `perform_pro_bundle_install`，本地没有 bundle，就去 console 拉 `flockspro` 渠道的 manifest、下载 bundle、在线跑 `uv sync` / `npm install` / `npm run build`，再 handoff 重启。升级时机器要能访问 portal、PyPI 和 npm registry（或配好的镜像 / 代理）；handoff 里 `uv sync` 或 npm 失败不会自动回滚，网络不通时服务会停在半路。冒烟测试在无网络容器里起一个本地的 console 替身（manifest 正常、bundle 地址 404）：开源包的实例越过离线拦截、去下载 bundle，下载失败后什么都没动；同一步换成离线 env 作对照，会在下载之前被拒绝。

下面说的是带 Pro bundle 的包。页面上"开始升级"调的还是 `perform_pro_bundle_install`；命令行是 `flocks update --pro-bundle [--no-restart]`，加 `--check` 只显示将安装的 bundle（来源、版本、release id、当前是否已装），不安装也不重启，也不碰 SQLite（session token 只读共享文件）；`FLOCKS_PRO_BUNDLE_DIR` 指着一个坏目录时直接报那个目录，不去问

Console。它先看 `FLOCKS_PRO_BUNDLE_DIR`（安装包写进 env 的 `/opt/flocks/bundle`）：目录里有 `manifest.json` 就用本地包，不下载；manifest 标了 `prebuilt: true` 且没带 `flocks/` 源码，就只做 `uv pip install --no-deps <wheel>`，跳过 `uv sync` 和 npm，写 marker 后走 handoff 的 Pro-only 重启。console 的 manifest 只是尽力拉一次：bundle 版本相同、且双方都带的 `core_version` / `flockspro_component_version` / `build_id` 都一致时，只把 `release_id` / `bundle_release_id` 合进 marker；其余字段一律以本地包为准（同一个 bundle 版本可以配不同 wheel 重打，不能因为版本号相同就把 console 那份的组件版本和 build id 记成已装）；拉不到、版本不同或身份冲突都不合并，也不影响安装。

`FLOCKS_UPDATE_CHANNEL` 被安装包固定成 `flockspro-offline-<core 版本>`，心跳和升级申请都会带上这个 `channel` 字段；console 给每个离线包版本开一个冻结渠道、发和包里同一个 release，在线客户的 `flockspro` 渠道不受影响。老客户端不带 `channel` 字段，console 按 `flockspro` 处理。

页面上“降级到 OSS”走 `perform_pro_bundle_downgrade`：卸掉 flockspro wheel 后的 handoff 重启在离线实例上（`FLOCKS_DEPLOY_MODE=offline`）带 `--prebuilt`，跳过 `uv sync` / `npm install` / `npm run build` 直接 `flocks start`；在线实例的降级 handoff 不变。

`prebuilt` 格式的 bundle 只能发到离线渠道。老版本客户端拿到没有 `flocks/` 目录的包会判成"不是 Pro bundle"，直接报"未找到 flockspro wheel"。

和原来的升级方式是什么关系

开源版的包装出来的实例，升级就是普通源码安装的在线路径，下面说的都是带 Pro bundle 的离线实例。离线实例的升级机制和在线安装是同一套代码（备份到 `~/.flocks/version`、`_replace_install_dir` 保留 `.venv`、marker、handoff 重启），只是"包从哪来"不同：Pro 组件来自包内 `bundle/`，核心升级靠新版 `.run` 覆盖执行。env 里的 `FLOCKS_DEPLOY_MODE=offline` 会让页面"检查更新 → 升级"、`flocks update` 和 `/api/update/apply` 在动任何文件之前就拒绝联网升级并提示用新 `.run`（否则会先替换源码再在 `uv sync` 上被 `UV_OFFLINE=1` 卡住，服务停在半路）。页面上 `update_allowed=false` 时显示"离线安装包部署"的提示，不是 Docker 那条；这块提示不看 `has_update`。核心版本检查（`edition=flocks`）在离线实例上根本不出网：`check_update` 直接返回本地版本、`latest` 为空、无错误——之前会挨个等 GitHub/Gitee 超时（30—45 秒），最后一级兜底还会在当前目录跑 `git tag`，在别的 git 仓库里执行 CLI 甚至能凑出一个假的“有新版本”。页面这时显示“离线部署，未查询远端版本”而不是“已是最新”；`flocks update` / `--check` 先打提示、再打当前版本，退出码 0，不问“是否使用中国镜像”。Pro 那条（`edition=flockspro`，走 console manifest）不受影响，console 是这类机器唯一允许访问的端点。

带 Pro bundle 的离线实例如果后来能联网、想改回在线升级：从 `/etc/flocks/flocks.env` 删掉 `FLOCKS_OFFLINE_INSTALL` / `UV_OFFLINE` / `npm_config_offline` / `FLOCKS_PRO_BUNDLE_DIR` / `FLOCKS_UPDATE_CHANNEL`，把 `FLOCKS_DEPLOY_MODE` 改成 `source`，`systemctl restart flocks`，之后就是普通源码安装的在线升级路径（GitHub/Gitee 源码包 + console `flockspro` 渠道），代码不用改。之后再跑带 Pro bundle 的 `.run` 会把这些开关写回来；Pro 还没激活的机器也可以直接用开源版的包重跑，安装脚本会自己切过去。反过来，`install.sh` 装的实例改用 `.run`：`.run` 是另一种部署形态（`/opt/flocks` 或 `<基目录>/flocks/app`、`flocks` 用户、`/var/lib/flocks/.flocks` 或 `<基目录>/flocks/data/.flocks`），旧实例的 `~/.flocks` 会按上面「旧数据怎么带过来」自动拷一份；旧实例要先停，端口还被它占着时安装脚本会报错退出，并提示先 `flocks stop`。

在线用户不受影响：新逻辑全部靠 `FLOCKS_PRO_BUNDLE_DIR`、`FLOCKS_DEPLOY_MODE=offline`、manifest 的 `prebuilt` 和 handoff 的 `--prebuilt` 参数触发，`service_manager` 在 `ps` 不可用时（没有这个命令，或 BusyBox 那种不认 `-p` / `-eo` 的 `ps`，总之调用失败）改读 `/proc`——最小化容器里 `flocks stop` 找不到守护进程会把旧后端留在端口上；`ps` 正常的机器走原来的路径，console 现在发的 bundle（带 `flocks/`）新代码照收，既有 updater 测试全部原样通过。

测试

```
uv run pytest tests/scripts/test_linux_offline_installer.py    # 语法、函数先定义后调用、无递归删除、模板键、两种升级方式渲染出的 env、dry-run（含端口沿用/覆盖、upgrade=online|offline）、源码导出不带本地密钥、已激活 Pro 的沿用/拒绝、firewalld 永久规则（假 firewall-cmd）、随机密码、旧数据迁移（选源顺序、多个候选时拒绝、源目录只读、路径改写、占用和空间检查、FLOCKS_ROOT 挪过盘时的 Pro 判断）；基目录：规则、派生路径、重跑找回原目录、拒绝搬家、relocate.py 的 scan/apply、makeself 头部按 makeself 的方式展开后跑
uv run pytest tests/updater/test_updater_prebuilt_bundle.py     # 本地 bundle / prebuilt 路径、离线降级 handoff 带 --prebuilt、console 身份只在完全匹配时借 release_id，以及默认路径不变的回归
uv run pytest tests/cli/test_update_command.py                  # 离线实例 flocks update 先给 .run 指引
cd webui && npx vitest run src/components/common/UpdateModal.test.tsx   # 离线提示不依赖 has_update
uv run pytest tests/cli/test_service_manager.py -k proc             # 没有 ps 时的 /proc 兜底
packaging/linux/smoke-test.sh Output/flocks-offline.run         # 模拟内网的容器里真装一遍（需要 docker，镜像默认 UBI 8.10，架构要和包一致）；按包里有没有 bundle 走两套检查
SMOKE_BASE=/data packaging/linux/smoke-test.sh Output/flocks-offline.run   # 同一套检查换成基目录布局
sudo packaging/linux/verify-install.sh --port 5173 --rerun /var/tmp/flocks-offline.run   # 在装好的真机上验收
```

`SMOKE_INIT=systemd` 要一个带 systemd、firewalld、chrony 的镜像。UBI 的源里没有 firewalld 和 chrony，可以在 `registry.access.redhat.com/ubi8/ubi-init:8.10` 上临时加一个 Rocky Linux 8 BaseOS 源（同一份 RHEL 8.10 源码的重编译版）只用来装这两个包，glibc 和 systemd 仍然是 Red Hat 的。

真机上 5173 已被别的服务占用时，安装和验收都可以换端口：`sudo FLOCKS_OFFLINE_PORT=5273 bash /var/tmp/flocks-offline.run`、`verify-install.sh --port 5273`。安装脚本发现端口被占且没有旧的 Flocks 安装会直接报错退出，不会去停别人的进程。

真机验收清单在 `webook_dev/docs/offline_install/` 的差距分析文档里（断外网只放行 portal/passport 的 CentOS Stream 9 x86_64，按客户说明书原文从 scp 跑到 Pro 激活、两次重启、重复执行 `.run`）。

在 Docker 里模拟客户内网

客户的服务器在内网里，按离线安装手册（《Flocks 离线安装操作说明》“需要 Pro 时”那张网络表）只给服务器放行 `portalflocks.threatbook.cn`（106.75.19.23，TCP 443）；`passport.threatbook.cn` 那几行是给管理员电脑浏览器登录用的，服务器访问不到。`smoke-test.sh` 默认就在这样的环境里跑，放行清单在 `intranet-allowlist.txt`，手册里的地址变了要同步改它（`tests/scripts` 里有用例对照手册的服务器那一行）。

做法：先起一个网关容器（alpine，装 iptables 和 dnsmasq，这一步要测试机能上网），被测容器用 `--network container:<网关>` 共用它的网络命名空间。在这个命名空间里：

- DNS 只认清单里的域名，并解析到手册给的 IP；其余域名一律 NXDOMAIN。
- 出站只放行回环、入站连接的回包、清单里的 IP 和端口，其余一律丢弃（和真防火墙一样表现为超时，不是立刻拒绝）；IPv6 全部丢弃。
- 端口发布在网关上，浏览器照样能打开被测容器里的 WebUI。`docker restart` 被测容器（模拟重启）不会丢掉这些规则。

```
packaging/linux/intranet-sim.sh up --name lab-gw --publish 15173:5173      # 起网关；--publish 可选
docker run -d --name lab --network container:lab-gw
        registry.access.redhat.com/ubi8/ubi:8.10 sleep infinity
packaging/linux/intranet-sim.sh check lab      # 在容器里核对：清单里的地址能通，其余域名、地址、端口都不通
packaging/linux/intranet-sim.sh render         # 看网关里实际执行的 DNS 和防火墙规则
docker rm -f lab && packaging/linux/intranet-sim.sh down --name lab-gw
```

`check` 逐条输出 PASS/FAIL：清单里的域名解析到手册 IP、TCP 端口能连、HTTPS 有应答；同一 IP 的其他端口不通；pypi.org、files.pythonhosted.org、github.com、registry.npmjs.org、mirrors.aliyun.com、www.baidu.com、passport.threatbook.cn 都解析不了；1.1.1.1、223.5.5.5、PyPI 的 CDN 地址和 passport 的两个手册 IP 都连不上；直接向 8.8.8.8、223.5.5.5、114.114.114.114 发 UDP DNS 查询没有应答。最后读网关里 IPv4 / IPv6 的 OUTPUT 规则：策略必须是 DROP，放行的只能是回环、回包和清单里的地址端口，测试机没有 IPv6 时探测不出来的问题靠这一步兜住。`smoke-test.sh` 在装包前、装完（安装脚本动过 firewalld）和模拟重启后各跑一次，任何一条不对整项算失败；清理时只删本次建的容器和网关，同名的旧网关（比如上次 `SMOKE_KEEP=1` 留下的）不会被误删。完全不联网的测试机用 `SMOKE_NETWORK=none`（网关装不了软件，内网模拟起不来）。